Variables

boxes boxes boxes boxes

Variables

```
1   name = "Neil Arthur Alaan"
2   print(name)
```

Variables are a fundamental part of programming; having a weak understanding of variables will likely lead to difficulties in programming

Models, blackboxing, and ways to think

Models

Using analogies, models, and pre-existing patterns to think about new ideas is a core strategy in learning

> Human brains are fundamentally **pattern matching machines**

And so having a clear model, a way of understanding, is a requirement to learning not just variables but programming as a whole

The box model for variables and data

Imagine a *box* and a *blob of stuff*.

The **variable is the box**; the **value is the blob** inside it

When you write

```
name = 20
```

You are *making a box*, putting a *label* on that box, and then *putting something* in that box

And when you write

```
5 < name
```

You are simply **finding** the box with the **label** *name*, and replacing the *name* with the **value**

```
5 < 20
```

You don’t have to use the box model

1. Sticky notes and values – the name is a sticky note you can move onto different values
2. A dry erase whiteboard spot – the variable is a labelled area, the value is what’s written in it
3. Movie character/acting role – the name is the role, different actors (values) can play it

It doesn’t matter what you use, as long as you use it consistently and build up on top of that

Breaking the model

There will come a time where your analogy *falls apart*

> Two labels on the same box:
> writing `a = b` does *not* copy the value,
> it puts a second label on the same box.
> Changing the contents through `a` also changes what `b` sees

Part of learning is slowly *refining* your model whenever it **breaks**, until it becomes closer and closer to how the real thing functions

This is an uncomfortable process, trying to understand something that will not compute in your own way of thinking

But pushing through that, and refining your own models, is what leads to actual *mastery*

Patterns and scaffolds

Common variable patterns

To more easily understand how variables work, we'll be discussing some of the more common patterns that show up

Where they're usually used, how they usually work, etc

Configuration bundle

Think of a game with a player that has

- `speed`
- `jumpForce`
- `mass`

By setting these variables at the very top of your script, it makes tweaking these values significantly easier since you don’t have to hunt through your code to find them

Configuration bundle (cont)

```
speed = 400
jumpForce = 600
mass = 5
gravity = 9.8


velocity.y += gravity * mass

if input == "left arrow":
    velocity.x = -speed

if input == "right arrow":
    velocity.x = speed

if input == "jump":
    velocity.y -= jumpForce
```

Counters

Another thing that variables are used for are for **counting** things

Every time the player touches a coin, the score should *go up* by `1`

We need

1. A place to *store* the count
2. A *starting* value
3. Something that *triggers* the increase
4. the *change* itself

Counters (cont)

```
score = 0

if player touches coin:
    score = score + 1
```

Step	Event	`score` value
Start	Game starts	0
1	Touch coin 1	1
2	Touch coin 2	2
3	Touch coin 3	3
…	…	…

Countdown (timer)

The opposite of counting up is counting down

Essentially the same as counting up, but just goes down instead

And we **stop** when we hit a target value

```
timeLeft = 30

while timeLeft > 0:
    sleep(1)
    timeLeft -= 1
```

Countdown (timer)

Step	Action	`timeLeft` value
Start	Initialize	30
1	Wait 1 sec, change by -1	29
2	Wait 1 sec, change by -1	28
…	…	…
30	Wait 1 sec, change by -1	**0 → stop**

Flags

Often we need a variable that stores the *current state* of something

either in the form of discrete states

- `state = "playing"`,
- `state = "paused"`,
- `state = "game over"`,
- `state = "menu"`

or checks

- `is_paused = False`
- `is_playing = True`

Flags (cont)

```
state = "playing"

while state == "playing":
    # play the game
    if player presses "pause":
        state = "paused"
```

or

```
is_playing = True
is_paused = False

while is_playing:
    # play the game
    if player presses "pause":
        is_playing = False
        is_paused = True
```

Flags (cont)

A flag variable doesn't count, it **switches** between values:

```
gameState:  menu  →  playing  →  game over
                ↑          ↓
                └─ paused ──┘
```

The variable acts as a **traffic light**

This replaces a tangled mess of `if` blocks with a single variable that controls the flow.

Or a collection of descriptive flags, like `is_paused`, `is_muted`, `is_on_ground` – each with a clearly defined meaning

Max trackers

How do you remember the highest score

Exactly the same way as setting the score (counting), but with an added check

1. A place to *store* the record
2. A *starting* value (usually 0 or a negative number)
3. A *comparison* each time the score changes
4. The *update* — only when the new score beats the record

```
highScore = 0

if score > highScore:
    highScore = score
```

The `if` only updates the record when it's beaten, so `highScore` always remembers the best

Swaps

As an open challenge, figure out a way to switch the values held by `x` and `y`

```
x = 10
y = 50
```

The goal output is

```
x = 50
y = 10
```

Rules: you can only use the variables `x`, `y`, and a helper variable – you may **not** type the values `10` or `50` directly

Swaps (reveal)

The classic approach uses a temporary variable to hold one value while the other is moved

```
temp = x
x = y
y = temp
```

And in Python there's a shortcut: you can swap in one line with **tuple unpacking**

```
x, y = y, x
```